

CelerLog: Fast Log Parsing via Dynamic Routing

Shiwen Shan[¶], Yintong Huo[‡], Minxing Wang[‡], Zhiying Wu[¶], Yuxin Su[¶], Zibin Zheng[¶]

[¶]Sun Yat-sen University, [‡]Singapore Management University

{shanshw,wuzhy95}@mail2.sysu.edu.cn

{ythuo,mxwang}@smu.edu.sg,{suyx35,zhibin}@mail.sysu.edu.cn

Abstract

Log parsing is a fundamental step for automated log analysis, which transforms raw log messages into structured formats. Existing syntax-based parsers struggle with complex logs because they lack semantic reasoning ability. Emerging LLM-powered semantic parsers achieve high accuracy but suffer from prohibitive latency and token costs because they apply semantic inference across all logs. Our key observation is that not all logs necessitate complex semantic understanding; a vast majority of logs exhibit repetitive patterns that can be extracted via straightforward statistical analysis. Driven by this insight, we propose CelerLog, a fast and effective log parser. CelerLog introduces a dynamic routing mechanism to classify logs into dense and sparse groups. Logs with strong statistical patterns (dense groups) are processed by an efficient statistical processor, whereas the sparse groups lacking such patterns are routed to an LLM for semantic inference. This hybrid strategy avoids unnecessary LLM invocations. Extensive experiments on 14 public datasets show that CelerLog achieves leading performance over state-of-the-art baselines and is 7.9× to 18.6× faster than LLM methods and up to 1.5× faster than Drain. Additionally, it reduces costs by decreasing token consumption by 80.2%–94.1% and LLM invocations by 86.4%–90.9%.

ACM Reference Format:

Shiwen Shan[¶], Yintong Huo[‡], Minxing Wang[‡], Zhiying Wu[¶], Yuxin Su[¶], Zibin Zheng[¶]. 2026. CelerLog: Fast Log Parsing via Dynamic Routing. In . ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/nnnnnnn>.

1 Introduction

Log messages are generated at runtime by developer-embedded logging statements to record system behaviors [6, 20, 36], serving as a crucial data source for automated tasks such as error diagnosis [38, 44, 51, 54] and anomaly detection [11, 12, 23, 40, 53]. As the prerequisite step for these analyses, log parsing transforms semi-structured raw logs into structured formats [25, 42, 50]. Typically, a log message comprises two main parts: the *log templates* (constant string literals describing event types) and the *log parameters* (dynamic runtime values). Figure 1 illustrates a workflow: logging statements (e.g., `Logger.info`) produce raw messages containing headers (e.g., timestamp) and bodies during execution. The parsing process then identifies variable parameters (e.g., `/etc/zookeeper/conf/zoo.cfg`) and extracts the static template (e.g., `Reading configuration from: <*>`) for downstream tasks.

Considering that the source code may not be available, existing parsing tools rely on code-free parsing [15, 56]. Over the decades, log parsing tools can be divided into two categories: syntax-based and semantic-based. (1) *Syntax-based parsers* employ heuristics or statistical features to identify constant parts from variable ones. While generally efficient, these parsers often fall short in parsing

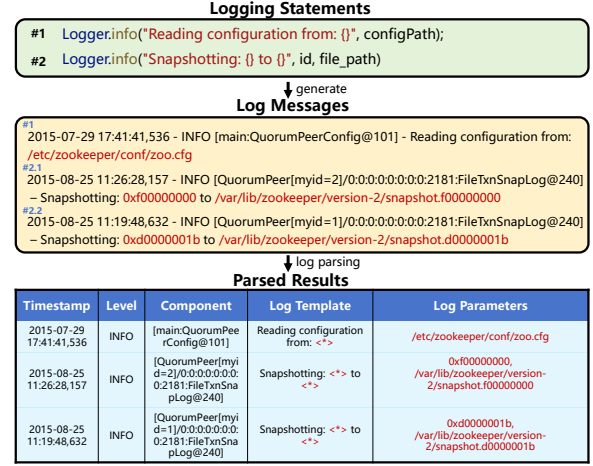


Figure 1: The general paradigm of log parsing.

effectiveness due to their lack of semantic awareness, struggling to handle complex log patterns accurately. (2) *Semantic-based parsers*, on the other hand, exploit semantic-aware models to distinguish dynamic parameters from static text. Recently, LLM-based parsers have emerged as a dominant solution in this category. By leveraging the powerful natural language understanding capabilities of LLMs, these methods achieve leading parsing accuracy, identifying parameters that traditional syntax rules might miss.

However, the heavy reliance of these semantic parsers on LLMs introduces prohibitive costs, such as processing latency and excessive token consumption, limiting their real-world practicality. Previous works attempted to mitigate this overhead through various strategies. For instance, LILAC [19] utilizes caching mechanisms to avoid redundant queries, while LUNAR [17] and LogBatcher [47] employ clustering to sample representative logs for query. Nevertheless, these methods face a fundamental bottleneck: they still depend on LLMs to process every distinct log group, failing to exploit the rich statistical patterns in large-scale log data.

Our investigation reveals a critical insight: **not all logs require semantic understanding for log parsing**. A majority of logs exhibit repetitive patterns with parameter variations, which can be easily extracted using statistical analysis. For example, in Figure 1, log messages #2.1 and #2.2 share the same template with different parameters. When such patterns occur with high frequency, statistical evidence alone is sufficient for automated parsing. Conversely, only logs with sparse patterns require further semantic reasoning of LLMs. For instance, log message #1 appears isolated without repetitive statistical clues. Therefore, it relies on LLMs to understand its semantics and identify its parameters.

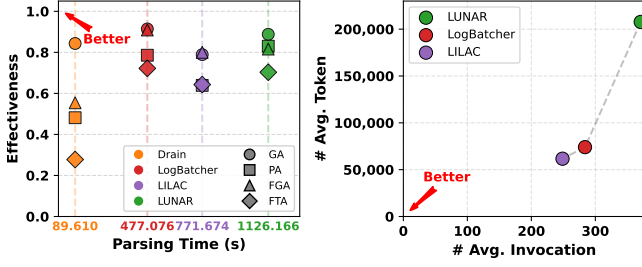


Figure 2: The parsing effectiveness-cost of log parsers. We use GPT-5.2 as the backbone LLM for the LLM-based parsers.

Driven by this insight, we propose a fast and effective log parser, namely CelerLog, with three components: a dynamic *router*, a simple yet fast *statistical processor*, and a semantic-aware *LLM-based processor*. The core design is a dynamic router that directs incoming logs to different processors. Specifically, it sends *dense log groups* (rich in statistical signals) to the *statistical processor* for instant syntax-based parsing, and routes *sparse log groups* (lacking statistical patterns) to the *LLM-based processor* for semantic analysis. To ensure efficient routing, we adopt a two-stage grouping phase and an anchor-based merging phase. We first identify skeleton groups and merge similar ones based on shared anchors. Groups that successfully merge with others are classified as *dense log groups*. The *statistical processor* then handles them by identifying columns with multiple distinct values as parameters. The remaining unmerged groups are deemed *sparse log groups*. Since these logs lack statistical variations, the *LLM-based processor* uses its semantic capability to identify variable types within the text. To further reduce costs, we process these sparse groups in batches and enable parallelism to boost speed.

Extensive experiments on 14 public datasets demonstrate the superiority of CelerLog. In terms of effectiveness, CelerLog outperforms state-of-the-art baselines by an average of 9.6% in grouping accuracy and 46.8% in parsing accuracy. Regarding efficiency, it achieves a 7.9–18.6 $\times$ speedup compared to LLM-based methods, and surpasses the fastest parser, Drain, by up to 1.5 $\times$ in the parallel version. Furthermore, CelerLog significantly reduces costs by decreasing token consumption by 80.2%–94.1% and LLM invocations by 86.4%–90.9%. Further ablation study, robustness analysis and sensitivity analysis confirm CelerLog’s robustness.

To conclude, our contributions are listed as follows:

- We identify that the bottleneck of existing LLM-based parsers is the unified semantic inference procedure to all logs, whereas a majority can be handled by statistical analysis.
- We introduce the concept of dense and sparse log groups, proposing a hybrid strategy that leverages statistical signals to minimize unnecessary LLM invocations.
- We propose CelerLog, an innovative log parser featuring a dynamic routing mechanism that directs logs to different processors, thereby maximizing efficiency and effectiveness.
- Comprehensive experiments show that CelerLog significantly outperforms state-of-the-art baselines in parsing effectiveness, token consumption, and processing time by orders of magnitude.

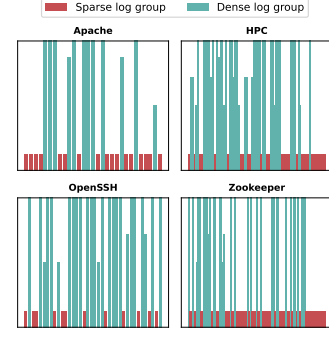


Figure 3: Distribution of dense and sparse log groups across four public datasets. The x-axis denotes unique templates, while the y-axis shows their corresponding message counts. Sparse groups are characterized by a single log message occurrence ($y = 1$).

2 Preliminaries

In this section, we discuss the trade-off between effectiveness and cost in log parsers, and introduce two distinct log categories: dense and sparse log groups.

2.1 The Effectiveness-Cost of Log Parsers

To comprehensively understand current log parsing tools, we evaluate them through two key dimensions: *parsing effectiveness* and *parsing cost*. Effectiveness refers to the accuracy of identifying log templates and parameters, while cost encompasses both parsing time and the financial expense of LLM invocations.

On one hand, syntax-based parsers prioritize efficiency. For instance, as shown in Figure 2 (left), Drain [16] completes parsing in 89 seconds on average. Its minimal latency has led to its wide adoption by industry leaders like IBM [34]. However, Drain significantly lags behind LLM-based methods in effectiveness, as it struggles to generalize to diverse or evolving log patterns without semantic understanding. On the other hand, while LLM-based parsers achieve superior accuracy, they incur prohibitive costs. As illustrated in Figure 2, methods like LogBatcher and LUNAR require 477 to 1,126 seconds, which is 5 $\times$ to 13 $\times$ slower than Drain. Beyond latency, the financial burden becomes a concern for deployment. In our experiments using GPT-5.2 [35], these methods consume over 50,000 tokens per dataset on average. When scaled to industrial volumes where systems generate petabytes of logs daily [25, 26], the costs increase linearly and thereby become financially unsustainable. The root cause of this inefficiency lies in their design principles: existing LLM-based parsers depend heavily on LLM reasoning for every distinct log pattern. Despite optimization strategies like caching or sampling, the fundamental dependency on LLMs for template extraction remains unchanged.

Consequently, although semantic parsers are accurate, this severe effectiveness-cost trade-off hampers their applicability. This observation prompts a critical question: *Is there an approach that bridges semantic and syntax parsers to exploit the strengths of both?*

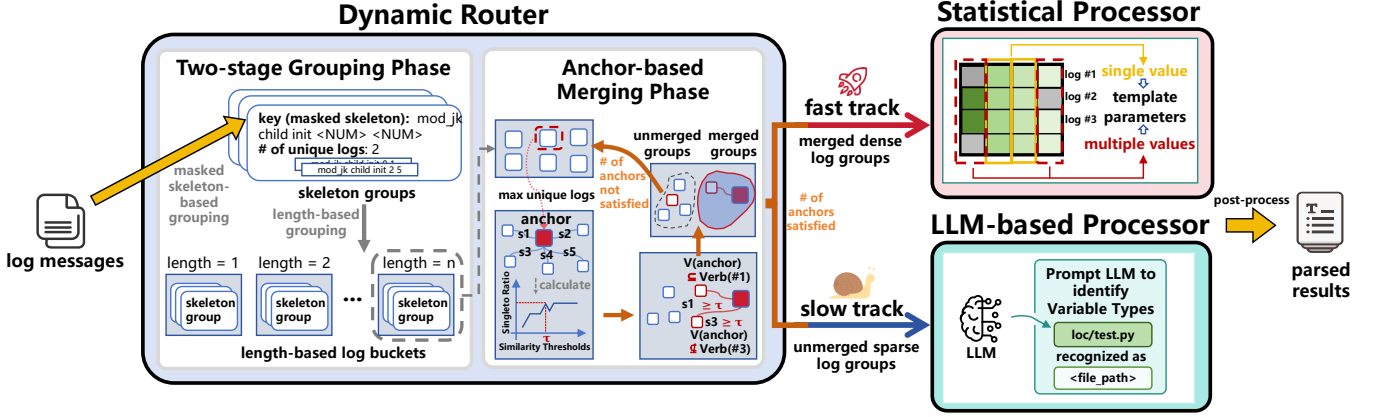


Figure 4: The overview of CelerLog.

2.2 Dense & Sparse Log Groups

To address the above question, we investigate the statistical distribution of log data. Our goal is to determine whether there exists a boundary that distinguishes logs suitable for syntax-based parsing from those requiring semantic inference.

Inspiration & Observation. We hypothesize that if a log template generates multiple unique messages with varying parameters, these variations provide sufficient statistical signals for pattern extraction without LLMs. Otherwise, the log template lacks such signals and necessitates semantic understanding. Our empirical analysis on 4 public datasets (Apache, HPC, OpenSSH and Zookeeper) [20] validates this hypothesis. As shown in Figure 3, we observe two distinct patterns in log distributions. The first type, represented by the tall teal bars, consists of numerous unique log messages derived from the same template. The variations in these messages (e.g., changing IP addresses) naturally highlight the parameter positions. The second type, represented by the short red bars, consists of an isolated log where the parameter values remain constant. This observation reveals that the disparate distribution of log groups enables a complementary parsing approach. For the log group rich in pattern variations, we can leverage the abundance of variable examples to perform lightweight statistical comparisons. For the isolated groups lacking such clues, we employ the semantic reasoning of LLMs. By tailoring the processing strategy to these distinctive categories, we can effectively bridge the trade-off between parsing cost and effectiveness.

Definitions. Based on the aforementioned observation, we formally categorize these groups into *dense log groups* and *sparse log groups*. A *dense log group* corresponds to the first type, defined as a collection of log messages belonging to the same template that exhibits dynamic patterns due to varying parameter values. Let a specific log $l(t, \vec{p})$ correspond to its log template t with its log parameters $\vec{p}$. The dense log group for specific log template $\mathcal{G}_D(t)$ is defined as

$$\mathcal{G}_D(t) = \{l(t, \vec{p}) | \vec{p} \in \mathbb{P}\}$$

where $\mathbb{P}$ is the parameter value space constituted by different parameter values. Conversely, a *sparse log group* exhibits a static appearance where parameter values remain constant. We define

the sparse log group $\mathcal{G}_S(t)$ as

$$\mathcal{G}_S(t) = \{l(t, \vec{p})\}$$

where the messages in $\mathcal{G}_S(t)$ share the identical parameters $\vec{p}$.

Lesson Learned: Log distributions exhibit two distinct patterns: *statistically rich dense groups* and *semantic-dependent sparse groups*. This division enables a hybrid strategy for log parsing that simultaneously achieves high effectiveness and low cost.

3 Methodology

CelerLog is designed to exploit the strengths of both fast syntax parsers and highly effective semantic parsers. As shown in Figure 4, CelerLog consists of a dynamic router and two parallel processors. First, the router processes log messages through a two-stage grouping phase to identify groups of logs with the same word skeleton (skeleton groups). Next, an anchor-based merging phase evaluates and merges similar skeleton groups. This step separates the logs into merged dense groups and unmerged sparse groups. The router then directs the dense groups to a statistical processor for efficient parsing, and the sparse groups to an LLM-based processor to identify complex variables.

3.1 Dynamic Router

Given log messages, the router aims to distinguish dense log groups from sparse ones. Crucially, this separation must be expedited to address latency concerns. We achieve this by clustering similar logs through a two-stage grouping phase and a merging phase.

3.1.1 Two-Stage Grouping Phase. This phase aims to identify logs with high lexical similarity, as they typically share the same template. We first abstract logs into masked skeletons by filtering out common variable tokens, which serve as the basic unit for all subsequent processing. Logs with identical skeletons form a skeleton group, and these groups are then further clustered by length. This two-stage, multi-granular process allows it to capture various lexical characteristics of log messages.

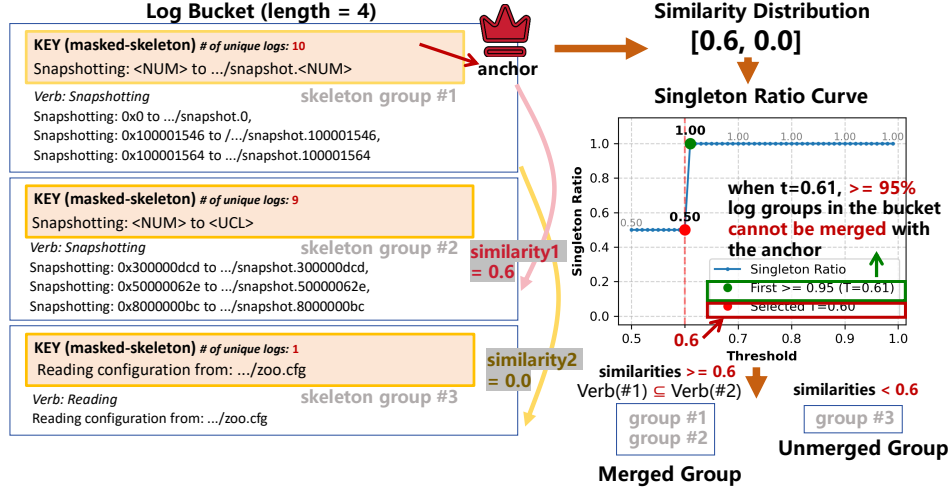


Figure 5: An example of the anchor-based merging phase of the Router.

Masked Skeleton-based Grouping. The first stage aims to construct fine-grained skeleton groups. Directly grouping raw logs often leads to excessive fragmentation due to the presence of dynamic variables. Therefore, we incorporate a preprocessing step using standard regular expressions, consistent with prior research [16, 17], to normalize the logs before parsing. Specifically, we mask five variable structures and replace them with designated tokens. These structures include pure numbers (<NUM>), mixed strings with clear boundaries (<CL>), mixed strings without clear boundaries (<UCL>), capitalized short strings (<BL>), and single letters with delimiters (<SL>). After masking, we extract the remaining text as skeletons. We then group the logs based on these masked skeletons. For each skeleton group, its masked skeleton serves as a unique key. This key represents all individual logs within the group. By processing through the key, we eliminate redundant computations and accelerate all the subsequent steps.

Length-based Grouping. The second stage applies a coarse grouping to the generated skeleton groups to prepare for efficient merging. While the strict first stage ensures high precision, it often splits logs of the same template into multiple skeleton groups, thus we need to combine these over-split groups for better approximation of the dense/sparse log groups. To limit the search space for potential matches, we aggregate the skeleton groups based on their length, since logs sharing a template typically have identical lengths [39]. Specifically, we determine the length of each skeleton group by counting the whitespace-separated tokens in its key. Finally, we place groups of the same length into corresponding log buckets.

3.1.2 Anchor-based Merging Phase. This phase aims to selectively merge highly similar skeleton groups within the same bucket. Groups that successfully merge together form dense log groups. Conversely, those that remain unmerged are classified as sparse log groups. The primary challenge lies in optimizing the matching efficiency. A naive exhaustive comparison among all groups leads to significant computational redundancy, especially as the number of unique skeletons scales. Moreover, employing a fixed similarity

Algorithm 1 Anchor-based Merging (per log bucket)

Require: Log bucket B
Ensure: Set of dense log groups G_d , set of sparse log groups G_s

```

1:  $G_d \leftarrow \emptyset, G_s \leftarrow \emptyset$ 
2: if log length of  $B$  is short or  $|B|$  is small then
3:   return  $G_d \leftarrow B$  {Directly route to the statistical processor}
4: end if
5:  $K \leftarrow \lfloor \alpha|B| \rfloor$  {Dynamically set top- $K$ }
6: Sort  $B$  by unique log count descending
7: while  $B \neq \emptyset$  and  $|G_d| < K$  do
8:    $A \leftarrow B[0]$  {Recursively select Anchor}
9:   Calculate similarities  $\Sigma = \{\text{PosJaccard}(A.\text{key}, x.\text{key}) \mid x \in B\}$ 
10:   $\tau \leftarrow \text{CalcThreshold}(\Sigma, \mathcal{P})$  {Set the threshold by  $\mathcal{P}$  metric}
11:   $\text{Matched} \leftarrow \{A\}$ 
12:  for all  $x \in B \setminus \{A\}$  do
13:    if  $\text{PosJaccard}(A.\text{key}, x.\text{key}) \geq \tau$  and  $\text{Verbs}(A.\text{key}) \subseteq \text{Verbs}(x.\text{key})$ 
14:      then
15:         $\text{Matched} \leftarrow \text{Matched} \cup \{x\}$ 
16:      end if
17:    end for
18:   $G_d \leftarrow G_d \cup \{\text{Merge}(\text{Matched})\}$ 
19:   $B \leftarrow B \setminus \text{Matched}$ 
20: end while
21:  $G_s \leftarrow B$  {Add remaining groups as sparse groups}
22: return  $G_d, G_s$ 

```

threshold lacks the flexibility needed to handle diverse log formats across different systems, often requiring extensive manual tuning.

To overcome these issues, we propose an anchor-based merging mechanism. Instead of pairwise comparisons, we select a few skeleton groups with the most unique logs as *bucket anchors*. These *anchors* generally represent the dominant portion of a dense log group. We then rapidly compare the remaining groups only against these anchors. This avoids pairwise checks and greatly accelerates the process. Further, to ensure robustness, we evaluate similarity using a dynamic threshold.

Algorithm 1 details the strategy. The input log bucket B consists of multiple skeleton groups. We first assess the complexity of the bucket. If the log length is short or the bucket contains few skeleton groups, we bypass the merging phase and route them directly to the statistical processor (Lines 2–3). We assume these buckets have

minimal fragmentation and stable structures. This avoids unnecessary computation and prevents excessive merging. For complex buckets, we dynamically calculate a limit K by setting K as $\lfloor \alpha|B| \rfloor$, where α is a predefined proportion parameter (Line 5). This adapts the limit to different bucket sizes. Next, we sort the skeleton groups in descending order based on their unique log counts (Line 6). We then iteratively select the top-ranked group as the anchor A and compute the similarity distribution Σ against the remaining groups using Position-aware Jaccard similarity (Lines 7–9). Subsequently, we derive an adaptive threshold τ from Σ to identify similar groups (Lines 10–11). We aggregate groups that satisfy both the similarity threshold τ and verb subset constraints. We then merge them into a dense log group in G_d (Lines 12–17). This iterative process continues until the bucket is empty or the number of identified dense log groups reaches the limit K (Lines 7–19). Finally, any remaining unmerged groups are categorized as sparse groups G_s (Line 20).

Similarity Distribution & Dynamic Similarity Threshold. As indicated in Lines 9–10 of Algorithm 1, we employ a dynamic thresholding mechanism based on the distribution of similarity scores.

For a selected anchor A , we first compute the Position-aware Jaccard Similarity against all other skeleton groups in the bucket to form a similarity distribution Σ . Unlike standard Jaccard Similarity [32], which treats logs as unordered bags of words, Position-aware Jaccard Similarity considers both the token value and its position index, thereby penalizing structural mismatches. To determine the optimal cut-off, we analyze the *Singleton Ratio Curve* of Σ , which tracks the proportion of unmerged groups (singletons) as the similarity threshold τ increases from 0.5 to 0.95. For instance, in Figure 5, the anchor compares against two groups with similarity scores of 0.6 and 0.0. At a threshold of 0.6, the first group merges while the second remains isolated. This yields one singleton out of two candidates, resulting in a ratio of 0.5 (1/2). Conversely, raising the threshold to 0.61 excludes both groups. The singleton count then equals the total count, pushing the ratio to 1.0 (3/3).

We use the singleton ratio to determine the similarity threshold for merging skeleton groups with an anchor dynamically. An anchor represents a dominant part of a dense log group because it contains the most unique logs. Therefore, it should naturally merge with other members of the same dense group. However, an overly strict similarity threshold causes a high proportion of candidate groups to fail this merge and remain as singletons. To ensure that valid members are not excluded, we set a predefined strict limit (i.e., $\mathcal{P}$) on the singleton ratio. Once the ratio reaches this limit, we revert to the immediately preceding similarity threshold. This prior value then serves as our final loose threshold. This strategy ensures that we capture the maximal valid merging boundary just before the similarity constraint becomes overly restrictive, thus preventing over-merging while maintaining high precision.

Lightweight Verb Checking. We incorporate a lightweight verb constraint to ensure semantic consistency. Some skeleton groups exhibit high structural similarity but perform completely different actions. To distinguish them, we extract verbs from the keys of both the anchor and the target skeleton group. We merge a skeleton group with the anchor only if its verb set contains all the verbs of the anchor. This guarantees that they share the same core action.

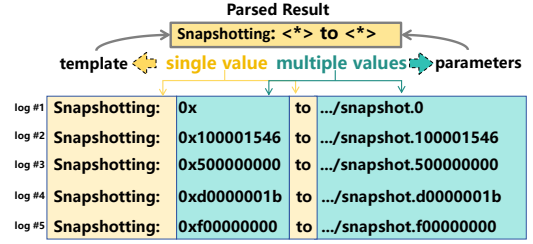


Figure 6: An example of the Statistical Processor.

Moreover, this check operates directly on the static keys rather than raw logs. Therefore, it adds minimal computational overhead.

Figure 5 shows an example of the anchor-based merging phase. In this example, the bucket contains three skeleton groups (i.e., Group #1, #2, and #3) with the same length of 4. The algorithm identifies Group #1 as the anchor because it contains the most unique logs. It then calculates the similarity distribution, yielding 0.6 for Group #2 and 0.0 for Group #3. According to the Singleton Ratio Curve, the ratio spikes to 1.0 (with $\mathcal{P} = 0.95$) at a threshold of 0.61. Therefore, the algorithm selects the preceding stable value of 0.6 as the adaptive threshold. Group #2 is successfully merged with the anchor as it satisfies both this similarity threshold and the verb consistency check. Conversely, Group #3 remains unmerged as a sparse group due to its distinct structure and action.

3.2 Statistical Processor

We design a straightforward yet efficient statistical processor to handle dense log groups identified by the router via a fast track. By leveraging robust statistical pattern extraction instead of expensive and potentially unstable LLM calls, it provides a highly efficient and consistent solution for dense log groups. To begin with, we analyze the value distribution at each position within the aligned log group to distinguish between unique values and multiple distinct values. We then directly mask the positions containing multiple distinct values as parameter positions. Finally, we apply a lightweight post-processing step following former practices [17, 19, 47] to refine the extracted templates and accommodate domain-specific formats. Figure 6 shows an example of the proposed statistical processor. As shown, the input consists of five aligned log entries. The processor identifies the tokens `Snapshotting:` and `to` as single values shared by all logs. These are retained as static template parts. Conversely, the hexadecimal strings (e.g., `0x0` and `0x100001546`) and file paths (e.g., `.../snapshot.0` and `.../snapshot.100001546`) exhibit multiple distinct values. These variable positions are replaced by wildcards (e.g., `<*>`). Consequently, the processor outputs the template `Snapshotting: <*> to <*>` and extracts the corresponding parameters.

3.3 LLM-based Processor

We propose the LLM-based processor to handle sparse log groups that lack sufficient signals for statistical analysis. Our task design for the LLM focuses on variable identification rather than full template generation to minimize hallucination risks. Specifically, we instruct the model to identify dynamic tokens and return them

You will be given a log message. Your task is to identify variable strings in the provided log message's context. Variables are dynamic runtime values that change between executions (e.g., numbers, hexadecimal, Job/Task IDs, URLs, file paths, service names, host names, versions, etc). Constants are static text that is fixed and never changes regardless of the execution. Output format per line: 'ID: [variable1<type>, variable2<type>]' Output the EXACT variables from the template. If no variables found, output 'ID: []'. Example Output: 1: [/var/etc/httpd/conf/<file_path>, com.tencent.health.manager.Service@32a6bf8<service>] 2: [blk_10983234328234<blk_id>, 192.168.0.1:8008<complex_ip>, root<user>]
Query

Figure 7: The prompt of the LLM-based processor.

as a list of exact variable strings. To ensure robustness, we process the LLM’s output by verifying that the identified variables are indeed present in the original log message. We then mask these validated variable substrings with a placeholder (e.g., <*>) to generate the final template. If the LLM fails to identify any variables or produces an invalid format, we roll back to the original log to prevent erroneous over-masking. Figure 7 shows the prompt template. The prompt comprises a task description, output constraints, and fixed-output examples following former works [17]. This structured design guides the model to produce stable and accurate results.

3.4 Parallel Processing

To address the scalability challenges in large-scale log parsing, we implement a two-level parallelization strategy within CelerLog. First, we exploit data parallelism during the routing phase by partitioning log buckets based on their token lengths. These buckets are processed independently across multiple CPU cores, allowing for concurrent anchor identification and similarity computation without cross-bucket dependencies. Second, for the LLM-based processor, we adopt asynchronous I/O to handle network latency. By batching API requests and utilizing non-blocking calls, we maximize throughput and ensure that the system remains responsive even when interacting with external model services. This hybrid approach effectively decouples computational tasks from I/O-bound operations, significantly reducing the overall parsing time.

4 Evaluation

We conduct evaluations to answer the following research questions (RQs):

- RQ1: How does CelerLog perform in log parsing?
- RQ2: How do different components contribute to CelerLog?
- RQ3: How sensitive is CelerLog to the parameter settings?
- RQ4: How robust is CelerLog with different backbone LLMs?

4.1 Dataset

We evaluate CelerLog on 14 publicly available log datasets [20, 55], aligning with prior studies [17, 19]. These datasets originate from a

Table 1: The statistics of the 14 public log datasets.

Systems	Datasets	# Logs	# Templates
Mobile systems	HealthApp	212,394	156
Server applications	OpenSSH	638,946	38
	Apache	51,977	29
Standalone software	Proxifier	21,320	11
Supercomputers	Thunderbird	16,601,745	1,241
	BGL	4,631,261	320
	HPC	429,987	74
Operating systems	Mac	100,314	626
	Linux	23,921	338
Distributed systems	Spark	16,075,117	236
	HDFS	11,167,740	46
	OpenStack	207,632	48
	Hadoop	179,993	236
	Zookeeper	74,273	89

diverse array of systems, including distributed systems, supercomputers, operating systems, and server applications. This collection covers a wide range of log scales and complexities, amounting to millions of log messages and thousands of unique templates, as summarized in Table 1. Such diversity provides a comprehensive benchmark for evaluating the parsing effectiveness and cost of CelerLog across different log datasets.

4.2 Baselines

Following prior work [17, 19], we compare CelerLog with six state-of-the-art log parsers, comprising three syntax-based approaches and three LLM-based semantic approaches. For the syntax-based parsers, we select Drain [16], AEL [22], and Brain [49]. These methods are widely recognized for their efficiency and strong performance among traditional syntax-based parsers. For the semantic-based approaches, we choose LILAC [19], LogBatcher [47], and LUNAR [17]. Since our method operates in an unsupervised manner without labeled data, we adopt the label-free variant of LILAC¹ (i.e., LILAC w/o ICL) for a fair comparison, as the original version relies on in-context learning with labeled examples. All baselines are evaluated using their official open-source implementations with default parameters to ensure reproducibility.

4.3 Metrics

Following prior practices [17, 19], we evaluate CelerLog from two primary aspects: parsing effectiveness and parsing cost.

4.3.1 Parsing Effectiveness. We use four standard metrics to assess parsing quality at both the message and template levels in alignment with existing works [17, 19, 20].

- **Grouping Accuracy (GA)** calculates the ratio of log messages assigned exactly to their correct ground truth clusters.
- **Parsing Accuracy (PA)** measures the ratio of messages where every token is correctly classified as a constant or a variable.
- **F1 score of Grouping Accuracy (FGA)** evaluates grouping performance at the template level. It calculates the harmonic mean of grouping precision and recall to address class imbalance.

¹Unless otherwise specified, we use LILAC to denote the label-free variant (LILAC w/o ICL) in the remainder of this paper.

Table 2: Effectiveness comparison with SOTA parsers (best: bold; second-best: underlined; '-': 24h timeout).

Method	Metric	Apache	BGL	Hadoop	HDFS	HealthApp	HPC	Linux	Mac	OpenSSH	OpenStack	Proxifier	Spark	Thunderbird	Zookeeper	Average
Syntax-based Log Parsers																
Drain	GA	1.000	0.919	0.921	<u>0.999</u>	0.862	0.793	0.686	0.761	0.707	0.752	0.692	0.888	<u>0.831</u>	<u>0.994</u>	0.843
	PA	0.727	0.456	0.546	0.569	0.312	0.721	0.112	0.381	0.586	0.020	<u>0.688</u>	0.559	0.219	0.844	0.481
	FGA	1.000	0.624	0.785	<u>0.935</u>	0.010	0.309	0.778	0.230	0.872	0.007	0.206	<u>0.861</u>	0.237	<u>0.904</u>	0.554
	FTA	0.517	0.204	0.394	0.478	0.004	0.147	0.262	0.070	0.487	0.002	0.176	0.448	0.072	0.627	0.278
AEL	GA	1.000	0.915	0.823	<u>0.999</u>	0.725	0.748	<u>0.917</u>	0.797	0.705	0.743	0.974	-	0.786	0.996	0.856
	PA	0.727	0.455	0.540	0.569	0.311	0.741	0.093	0.269	0.364	0.020	0.677	-	0.166	0.842	0.444
	FGA	1.000	0.587	0.117	0.764	0.008	0.201	0.812	0.793	0.689	0.682	0.667	-	0.116	0.788	0.556
	FTA	0.517	0.176	0.060	0.494	0.003	0.133	0.259	0.211	0.333	0.141	0.417	-	0.036	0.475	0.250
Brain	GA	0.997	0.940	0.503	0.960	0.865	0.800	0.790	0.808	0.663	1.000	0.521	0.838	0.792	0.993	0.819
	PA	0.262	0.435	0.123	0.722	0.308	0.662	0.067	0.324	0.261	0.149	0.687	0.556	0.199	0.818	0.398
	FGA	0.933	0.756	0.511	0.759	0.864	0.400	0.749	0.737	0.759	1.000	0.737	0.207	0.748	0.798	0.711
	FTA	0.433	0.179	0.188	0.437	0.411	0.184	0.230	0.277	0.299	0.312	0.421	0.086	0.260	0.546	0.305
LLM-based Log Parsers																
LILAC	GA	1.000	0.880	0.915	1.000	0.999	<u>0.871</u>	0.641	0.750	0.672	0.554	0.059	0.892	0.820	0.993	0.789
	PA	0.992	0.940	0.811	0.569	0.582	0.979	0.642	0.532	0.365	0.496	0.189	<u>0.874</u>	0.580	0.394	0.639
	FGA	1.000	<u>0.886</u>	0.913	0.658	0.978	0.922	0.739	0.815	0.708	<u>0.949</u>	0.062	0.863	0.811	0.900	0.800
	FTA	0.816	0.784	0.749	0.411	0.830	0.828	0.567	0.539	0.503	0.841	0.211	0.695	0.543	0.688	0.643
LogBatcher	GA	0.997	0.947	0.929	0.999	0.942	0.864	0.832	0.892	0.749	1.000	1.000	0.973	0.684	0.993	0.914
	PA	0.968	0.909	0.736	0.947	0.767	0.883	0.627	0.603	0.702	0.933	1.000	0.684	0.420	0.827	0.786
	FGA	0.918	0.903	0.870	0.968	<u>0.950</u>	<u>0.859</u>	<u>0.863</u>	<u>0.845</u>	<u>0.909</u>	1.000	1.000	0.829	<u>0.829</u>	0.978	0.909
	FTA	0.721	0.772	0.662	0.731	0.785	<u>0.817</u>	<u>0.667</u>	0.486	0.701	0.771	1.000	0.596	<u>0.551</u>	0.844	0.722
LUNAR	GA	0.999	0.946	0.938	0.916	0.992	0.907	0.706	0.874	0.761	0.968	0.669	0.906	0.860	0.993	0.888
	PA	0.914	0.970	0.897	0.960	0.944	0.922	<u>0.742</u>	<u>0.619</u>	<u>0.703</u>	0.892	0.680	0.947	0.584	<u>0.850</u>	<u>0.830</u>
	FGA	0.978	0.773	0.692	0.892	0.932	0.753	0.827	0.768	0.911	0.566	0.775	0.853	0.827	0.885	0.816
	FTA	0.728	0.721	0.562	0.904	0.731	0.814	0.608	0.506	<u>0.911</u>	0.508	0.864	0.658	0.533	0.792	0.703
Our Proposed Method (Hybrid)																
CelerLog	GA	0.997	0.944	0.919	1.000	0.984	0.861	0.940	0.916	0.780	0.960	<u>0.989</u>	0.889	0.815	0.988	0.927
	PA	0.997	0.937	0.768	1.000	<u>0.928</u>	<u>0.932</u>	0.857	0.632	1.000	<u>0.894</u>	1.000	0.855	0.451	0.993	0.875
	FGA	0.944	0.836	<u>0.892</u>	0.894	0.936	0.686	0.867	0.867	0.911	0.936	<u>0.870</u>	0.842	0.862	0.882	<u>0.873</u>
	FTA	0.876	0.742	<u>0.692</u>	<u>0.830</u>	0.861	0.705	0.747	0.541	0.962	<u>0.830</u>	<u>0.957</u>	<u>0.668</u>	0.592	<u>0.816</u>	0.773

• **F1 score of Template Accuracy (FTA)** evaluates both grouping correctness and text accuracy. It calculates the harmonic mean of template precision and recall. A template is correct only if its grouping and text perfectly match the ground truth.

4.3.2 Parsing Cost. To assess the efficiency and economic feasibility of our approach, we employ three cost-related metrics.

• **Parsing time** records the total end-to-end execution time. This includes preprocessing, model inference, and post-processing.

• **Token consumption** measures the total number of tokens processed by the LLM. It is a direct proxy for the economic cost.

• **LLM invocations** counts the total number of API calls made to the LLM service. Reducing these calls minimizes network overhead and prevents rate limits.

4.4 Environment and Implementation

All experiments are conducted on a MacBook Pro equipped with an Apple M4 Pro processor with 12 cores and 24GB of unified memory, running macOS Sequoia. We employ GPT-5.2-2025-12-11 [35] as the default LLM accessed via the standard OpenAI API interface [3]. To ensure reproducibility and eliminate randomness, the temperature parameter is set to 0. We use Spacy [2] with NLTK WordNet [1, 5] for lightweight verb validation. Regarding the hyperparameters, we configure the default dynamic similarity threshold using the P95 criterion (i.e., $\mathcal{P} = 0.95$) based on the singleton ratio curve, and set the top- k anchors dynamically to half of the log bucket length (i.e., $k = \lfloor |B|/2 \rfloor$ with $\alpha = 1/2$), which is further analyzed in our sensitivity study. For the parallel version of CelerLog, we utilize 8 CPU cores to process log buckets concurrently. In contrast,

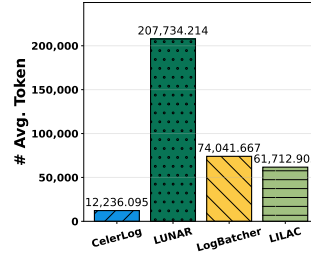


Figure 8: Average LLM token consumption of LLM-based log parsers and CelerLog. CelerLog achieves the lowest token consumption.

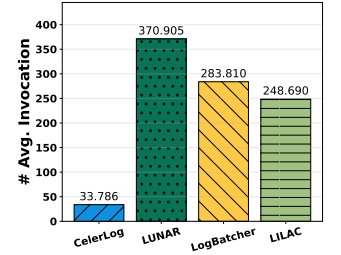


Figure 9: Average LLM invocations for LLM-based log parsers and CelerLog. CelerLog achieves the lowest invocations.

the non-parallel version operates sequentially on a single core to provide a baseline for efficiency comparison.

4.5 RQ1: Comparison Study

We evaluate CelerLog’s performance against state-of-the-art baselines in terms of parsing effectiveness and cost. On average, 98.7% of logs are identified as dense groups, while only 1.3% fall into sparse groups. Table 2 presents the comparison of CelerLog against syntax-based parsers (Drain, AEL, Brain) and LLM-based parsers (LILAC, LogBatcher, LUNAR). Syntax-based methods generally struggle with complex log patterns, as evidenced by Drain’s lower PA of 0.481. While LLM-based approaches leverage semantic understanding to improve accuracy, they often lack consistency in

Table 3: Parsing time (s) comparison with SOTA log parsers (best: bold; second-best: underlined; ‘-’: 24h timeout).

Dataset	Syntax-based			LLM-based			Proposed	
	Drain	AEL	Brain	LILAC	LogBatcher	LUNAR	CelerLog non-parallel	parallel
Apache	1.094	1.971	1.079	61.125	51.991	56.458	8.507	3.932
BGL	98.422	<u>58.906</u>	144.249	1834.448	573.258	1519.758	97.040	28.572
Hadoop	4.304	28.746	<u>8.020</u>	546.029	417.418	946.542	69.583	10.740
HDFS	<u>285.788</u>	658.768	1325.486	368.709	118.989	1448.672	305.582	358.084
HealthApp	3.747	141.797	28.364	267.664	245.609	261.333	43.880	<u>8.583</u>
HPC	8.632	9.739	<u>7.486</u>	175.825	144.696	288.611	18.480	5.542
Linux	0.600	1.833	<u>0.641</u>	659.839	620.941	545.985	143.284	18.656
Mac	<u>3.271</u>	5.306	2.938	1259.866	1243.351	2354.536	174.201	22.673
OpenSSH	<u>15.976</u>	114.593	22.121	78.295	68.868	139.490	16.763	12.063
OpenStack	20.504	11.794	8.253	121.632	81.821	391.994	18.952	<u>9.147</u>
Proxifier	0.769	1.094	<u>0.537</u>	29.232	20.811	24.474	0.577	0.588
Spark	296.517	-	3584.402	892.430	536.702	2405.993	<u>274.546</u>	259.544
Thunderbird	513.338	2037.152	8722.304	4339.079	2414.931	5233.850	<u>345.401</u>	104.551
Zookeeper	1.571	<u>1.658</u>	51.395	169.257	139.679	148.625	22.613	4.165
Average	<u>89.610</u>	> 236.412	993.377	771.674	477.076	1126.166	109.958	60.489
Speedup	1.5×	> 3.9×	16.4×	12.8×	7.9×	18.6×	1.8×	-

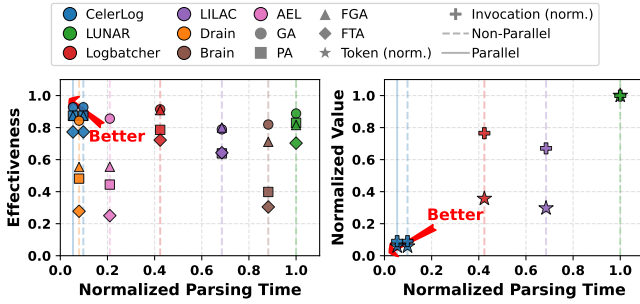


Figure 10: Performance and cost trade-offs of log parsers.
Left: Top-left is optimal (higher effectiveness, lower time).
Right: Bottom-left is optimal (lower resources, lower time).
 Values are scaled to $[0, 1]$ relative to the maximum observed.

fine-grained metrics. CelerLog outperforms all baselines across most datasets, achieving the highest GA of 0.927 and PA of 0.875. Notably, CelerLog surpasses the best-performing LLM-based baseline, LogBatcher, by a significant margin in PA (0.875 vs. 0.786).

Beyond accuracy, parsing efficiency is critical for practical deployment. Table 3 details the parsing time for all methods. Pure LLM-based methods suffer from high latency due to massive model inference, with LUNAR and LILAC requiring 1126.166 seconds and 771.674 seconds on average, respectively. In contrast, CelerLog (parallel) drastically reduces the average parsing time to 60.489 seconds, achieving an 18.6× speedup over LUNAR. Remarkably, CelerLog is even (1.5×) faster than the widely-deployed parser Drain (89.610 seconds). In terms of economic cost, Figure 8 and Figure 9 highlight the resource consumption. Existing LLM-based parsers incur heavy token usage. For instance, LUNAR consumes approximately 207,734 tokens on average. CelerLog reduces this consumption to merely 12,236 tokens, representing a huge reduction. Similarly, Figure 9 shows that CelerLog requires only 33.786

LLM invocations on average, whereas competing methods like LUNAR require over 370 invocations.

Effectiveness vs. Cost Trade-off. Figure 10 visualizes the balance between parsing effectiveness and parsing cost. The left plot demonstrates that CelerLog resides in the top-left region, indicating superior parsing effectiveness with minimal time overhead. The right plot confirms that CelerLog achieves the lowest normalized cost while maintaining high parsing speed. Consequently, CelerLog establishes a new SOTA Pareto frontier, offering a robust solution that is both highly accurate and cost-effective for large-scale log parsing.

Answer to RQ1: CelerLog not only achieves leading performance and runs tenfold faster (7.9×–18.6×) than baselines, but also significantly reduces costs by decreasing token consumption by 80.2%–94.1% and LLM invocations by 86.4%–90.9%.

4.6 RQ2: Ablation Study

To answer the question, we conduct an ablation study by comparing the full model with three variants: (1) **w/o router**, which randomly assigns logs to either the statistic or LLM processor instead of using the proposed routing mechanism; (2) **w/o statistical processor**, which selects the top three most frequent log messages from a dense group for LLM querying, replacing the statistical processor; and (3) **w/o LLM processor**, which removes the LLM component entirely. Table 4 presents the parsing effectiveness and cost comparison for these variants.

As the results indicate, the router is critical for balancing efficiency and effectiveness. Removing the router causes a severe degradation in parsing effectiveness, with GA dropping by 24.9% and PA by 10.7%. More importantly, the operational cost skyrockets: token consumption and LLM invocations increase by 331.6% and 339.1%, respectively. This indicates that the router effectively identifies and routes dense and sparse log groups, ensuring that expensive LLM resources are reserved only for sparse log groups.

Table 4: Ablation study of components of CelerLog. We use the non-parallel version to ensure clearer parsing time comparisons.

CelerLog	Effectiveness				Cost		
	GA	PA	FGA	FTA	Time (s)	# Token	# Invocation
Full	0.927	0.875	0.873	0.773	109.958	12,236.095	33,786
w/o router	0.696 (↓24.9%)	0.781 (↓10.7%)	0.676 (↓22.6%)	0.637 (↓17.6%)	325.958 (↑196.4%)	52,814.071 (↑331.6%)	148,357 (↑339.1%)
w/o statistical proc.	0.916 (↓1.2%)	0.622 (↓28.9%)	0.873 (–)	0.604 (↓21.9%)	270.748 (↑146.2%)	43,696.071 (↑257.1%)	181,143 (↑436.2%)
w/o LLM proc.	0.908 (↓2.1%)	0.815 (↓6.9%)	0.680 (↓22.1%)	0.473 (↓38.8%)	52.308 (↓52.4%)	–	–

Similarly, the statistical processor plays a crucial role in ensuring stability and efficiency when handling dense log groups. The *w/o statistical processor* variant results in a 28.9% decrease in PA and a 257.1% surge in token usage. This significant drop in effectiveness suggests that relying solely on the LLM for high-frequency logs introduces instability, primarily due to LLM hallucinations where parameters are often misidentified or over-extracted. By leveraging statistical patterns for dense groups, CelerLog mitigates these generative errors and achieves robust parsing results with substantially lower computational cost. As for the LLM processor, it provides the critical semantic understanding required for high-precision parsing. While the *w/o LLM processor* variant reduces parsing time by 52.4%, it causes a substantial degradation in fine-grained metrics, with FTA dropping by 38.8% and FGA by 22.1%. This result confirms that traditional syntax-based approaches are insufficient for capturing the semantic complexity of modern system logs, highlighting the indispensable role of the LLM in achieving the superior accuracy of CelerLog.

Answer to RQ2: Every component in CelerLog is essential, as the router maximizes efficiency, the statistical processor delivers stable templates, and the LLM-based processor provides semantic awareness for sparse log groups.

4.7 RQ3: Sensitivity Analysis

4.7.1 Dynamic Similarity Threshold. Figure 11 presents the parsing effectiveness and cost under different percentile settings for the dynamic similarity threshold (from P80 to P99). The results demonstrate that CelerLog is highly insensitive to this hyperparameter. Across the entire range, both effectiveness metrics and efficiency metrics remain remarkably stable with minimal fluctuations. This stability confirms the robustness of our singleton ratio curve-based approach, which adaptively determines the optimal cut-off for each log bucket rather than relying on a rigid global threshold. Consequently, we select P95 as the default setting to ensure a high safety margin against over-merging while maintaining excellent clustering performance.

4.7.2 Top-k Selection. Figure 12 illustrates the impact of different top-k anchor selection strategies on parsing effectiveness and cost. We compare two settings: (1) *Fixed setting*, where k is a constant integer (e.g., $k = 1, 3, 5, 10$); and (2) *Dynamic setting*, where k is a fraction of the log bucket capacity (e.g., $1/4, 1/3, 1/2, 2/3, 3/4$).

Sensitivity of Fixed Settings. As shown in the right part of Figure 12, the fixed setting exhibits high sensitivity and volatility. A small fixed k (e.g., $k = 1$) fails to capture sufficient anchors for complex buckets, leading to a surge in sparse log groups. This forces the downstream LLM processor to handle a significantly larger volume of logs, resulting in a dramatic spike in parsing time and

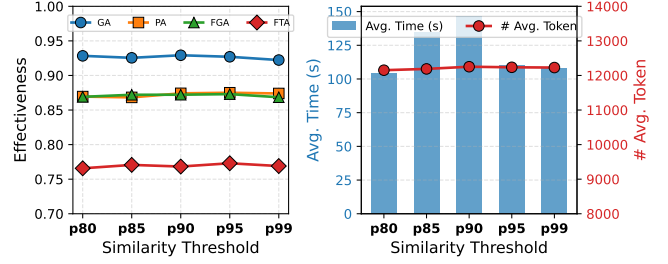


Figure 11: The parsing results under different similarity thresholds. The performance and cost remain stable across various percentile settings, demonstrating the robustness of our dynamic thresholding mechanism.

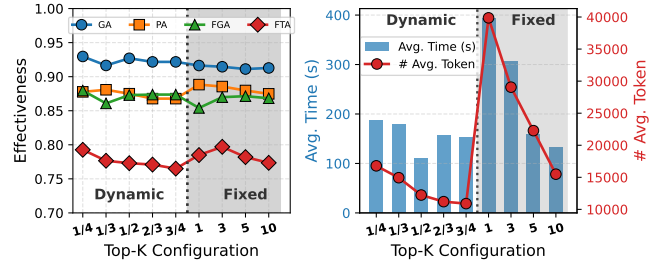


Figure 12: The parsing results under different top-k settings. The dynamic settings (fractions) exhibit significantly better stability and cost-effectiveness compared to the fixed settings (integers), with the $1/2$ ratio offering the optimal trade-off.

token consumption (e.g., average time > 350 s for $k = 1$). Conversely, a large fixed k increases the risk of selecting low-quality anchors, potentially causing over-merging and degrading accuracy.

Robustness of Dynamic Settings. In contrast, the dynamic setting (left part of Figure 12) demonstrates superior stability and generalization. By scaling k proportionally to the bucket size, CelerLog maintains a consistent balance between identifying dense groups and filtering sparse ones. Specifically, increasing the ratio from $1/4$ to $1/2$ significantly reduces computational cost as more logs are successfully routed to the efficient statistical processor. However, further increasing the ratio beyond $1/2$ yields diminishing returns in efficiency while slightly risking over-merging (evident in the slight drop in FTA at $3/4$). Therefore, to strike the optimal balance between effectiveness and cost-effectiveness, we adopt the dynamic setting with a ratio of $1/2$ as our default configuration.

Table 5: CelerLog’s performance with different LLMs.

Model	GA	PA	FGA	FTA
Default				
GPT-5.2	0.927	0.875	0.873	0.773
Large Language Models				
Gemini-3-pro	0.928	0.876	0.877	0.772
Qwen3-Max	0.924	0.868	0.870	0.764
Claude-sonnet-4.5	0.924	0.872	0.871	0.767
Small Language Models				
GPT-5-mini	0.926	0.870	0.874	0.768
Gemini-2.5-flash-lite	0.924	0.870	0.868	0.762
Qwen3-32b	0.924	0.862	0.865	0.756
Qwen3-14b	0.915	0.861	0.870	0.749
Qwen3-8b	0.915	0.863	0.863	0.744

Answer to RQ3: CelerLog demonstrates minimal sensitivity to parameter settings. Both the dynamic similarity threshold and dynamic top- k selection mechanisms ensure stable performance, with overall accuracy fluctuations of less than 1.5% across a wide range of parameter settings.

4.8 RQ4: Robustness Analysis

To evaluate the generalizability and robustness of CelerLog, we conduct extensive experiments across a diverse spectrum of backbone models, ranging from state-of-the-art Large Language Models like Gemini-3-pro to more efficient Small Language Models such as Qwen3-8b. Table 5 shows the results. The results demonstrate that CelerLog maintains remarkably stable performance regardless of the underlying model’s scale. For instance, the variance in GA remains minimal, with Gemini-3-pro achieving 0.928 and the smaller Qwen3-8b still maintaining a high 0.915. This stability stems from our core hybrid architecture, which integrates statistical heuristics with LLM-based reasoning. By offloading deterministic patterns to the statistical component, we reduce the burden on the LLM, ensuring that even smaller models can deliver results comparable to their larger counterparts. Furthermore, the results also confirm the pivotal role of the proposed router in this robustness. It effectively steers log entries to the most appropriate processing path, preventing the system from over-relying on the reasoning capabilities of any single backbone. Even when utilizing Gemini-2.5-flash-lite, CelerLog achieves an FTA of 0.762, which is nearly identical to the 0.772 produced by the much larger Pro version. These findings confirm that CelerLog is not vertically dependent on a specific proprietary model but is a framework capable of delivering consistent, high-quality parsing results across various models.

Answer to RQ4: CelerLog demonstrates robustness across diverse backbone LLMs, maintaining stable performance from large language models to smaller scales.

5 Threats to Validity

Internal Threats. The primary internal threat arises from the inherent randomness of the LLM used in the slow track. To mitigate this stochastic behavior, we set the temperature parameter to

zero. This ensures the model generates deterministic outputs for identical inputs. Additionally, we repeated all experiments three times to average out any remaining fluctuations. Another internal threat involves the implementation bias of baseline methods. We addressed this by adopting the official open-source implementations for all comparison partners. We also maintained consistent hyperparameters to ensure a fair evaluation environment.

External Threats. A major external threat is the potential data leakage during the LLM training phase. Since LLMs consume vast amounts of public data, they might memorize log templates. However, our approach tasks the model with variable identification rather than full template generation. This specific instruction reduces the reliance on memorized patterns. Furthermore, the quality of the evaluation datasets impacts generalizability. To counter this, we conducted evaluations on the extensive LogHub-2.0 benchmark. These datasets span various system types and verify that our method generalizes well across different domains.

6 Related Work

Log Parsing. Log parsing aims to transform raw log messages into structured templates by identifying dynamic parameters [17, 19, 24, 37, 41]. Existing methods are primarily categorized into syntax-based [9, 10, 12, 16, 30] and semantic-based approaches [18, 24, 27, 29, 52]. The syntax-based parsers, such as Drain [16] and LogMine [13], rely on heuristic rules or tree structures to extract frequent patterns. While efficient, they often lack flexibility when dealing with complex or evolving logs. Semantic-based methods usually utilize neural networks to capture log meanings. However, they depend heavily on labeled datasets, which are expensive to annotate. Recently, LLMs have achieved state-of-the-art performance in this task [17, 19, 29, 47]. Despite their high accuracy, applying LLMs to every log message incurs prohibitive computational and financial costs (i.e., longer parsing time, higher token consumption, and more invocations). To address this, our approach leverages the insight that not all logs require heavy LLM inference. We identify and categorize logs into dense and sparse groups. A dynamic router then directs dense groups to an efficient statistical processor and only forwards sparse groups to the LLM. This hybrid design effectively boosts high parsing accuracy with cost efficiency.

Automated Log Analytics. Log parsing serves as a critical step for downstream tasks [4, 7, 28], such as anomaly detection [11, 14, 31, 53], root cause analysis [8, 43, 45], and system error localization [38, 44]. In the era of LLMs, many LLM-based tools targeted at automated log analytics have been proposed [21, 33, 38, 46, 48]. These powerful tools excel at understanding complex system behaviors and diagnosing unseen errors. However, their practical deployment still relies on high-quality and structured log inputs. Our cost-efficient parser ensures that these downstream tasks receive accurate templates without excessive computational overhead.

7 Conclusion

In this paper, we propose CelerLog, an efficient and cost-effective log parser. Our work is driven by the insight that not all logs require complex semantic understanding. We design a dynamic routing mechanism to classify logs into dense and sparse groups. Since dense groups have clear statistical signals, we direct them to a fast

statistical processor. This ensures stability and eliminates hallucination risks without incurring LLM costs. Conversely, sparse groups lack these patterns. Therefore, we route them to an LLM processor for semantic analysis. Extensive experiments on public datasets show the superiority of CelerLog. It achieves superior grouping and parsing accuracy compared to current baselines. Furthermore, CelerLog parses logs much faster than existing LLM methods and syntax parsers. Finally, it reduces token consumption by orders of magnitude, making it highly practical for real-world applications.

References

- [1] 2001. NLTK Project. <https://www.nltk.org/howto/wordnet.html>. Online; Accessed: 2026-03-16.
- [2] 2016. Industrial-Strength Natural Language Processing. <https://spacy.io/>. Online; Accessed: 2026-03-16.
- [3] 2026. OpenAI API. <https://openai.com/blog/openai-api>. Online; Accessed: 2026-03-16.
- [4] Shan Ali, Chaima Boufaied, Domenico Bianculli, Paula Branco, Lionel Briand, and Nathan Aschbacher. 2023. An Empirical Study on Log-based Anomaly Detection Using Machine Learning. *arXiv preprint arXiv:2307.16714* (2023).
- [5] Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural language processing with Python: analyzing text with the natural language toolkit*. "O'Reilly Media, Inc."
- [6] Boyuan Chen, Jian Song, Peng Xu, Xing Hu, and Zhen Ming Jiang. 2018. An automated approach to estimating code coverage measures via execution logs. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. 305–316.
- [7] Zhuangbin Chen, Jinyang Liu, Wenwei Gu, Yuxin Su, and Michael R Lyu. 2021. Experience report: Deep learning-based system log analysis for anomaly detection. *arXiv preprint arXiv:2107.05908* (2021).
- [8] Tianyu Cui, Ruowei Fu, Changchang Liu, Yuhe Ji, Wenwei Gu, Shenglin Zhang, Yongqian Sun, and Dan Pei. 2025. AetherLog: Log-based Root Cause Analysis by Integrating Large Language Models with Knowledge Graphs. In *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 49–60.
- [9] Hetong Dai, Heng Li, Che-Shao Chen, Weiye Shang, and Tse-Hsun Chen. 2020. Logram: Efficient Log Parsing Using n n-Gram Dictionaries. *IEEE Transactions on Software Engineering (TSE)* 48, 3 (2020), 879–892.
- [10] Hetong Dai, Yiming Tang, Heng Li, and Weiye Shang. 2023. PILAR: Studying and Mitigating the Influence of Configurations on Log Parsing. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 818–829.
- [11] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 1285–1298.
- [12] Qiang Fu, Jian-Guang Lou, Yi Wang, and Jiang Li. 2009. Execution anomaly detection in distributed systems through unstructured log analysis. In *2009 ninth IEEE international conference on data mining (ICDM)*. IEEE, 149–158.
- [13] Hossein Hamooni, Biplob Debnath, Jianwu Xu, Hui Zhang, Guofei Jiang, and Abdullah Mueen. 2016. Logmine: Fast pattern recognition for log analytics. In *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management (CIKM)*. 1573–1582.
- [14] Minghua He, Tong Jia, Chiming Duan, Huaqian Cai, Ying Li, and Gang Huang. 2025. Weakly-supervised log-based anomaly detection with inexact labels via multi-instance learning. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2918–2930.
- [15] Pinjia He, Jieming Zhu, Shilin He, Jian Li, and Michael R Lyu. 2016. An evaluation study on log parsing and its use in log mining. In *2016 46th annual IEEE/IFIP international conference on dependable systems and networks (DSN)*. IEEE, 654–661.
- [16] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R Lyu. 2017. Drain: An online log parsing approach with fixed depth tree. In *2017 IEEE international conference on web services (ICWS)*. IEEE, 33–40.
- [17] Junjie Huang, Zhihan Jiang, Zhuangbin Chen, and Michael Lyu. 2025. No more labelled examples? an unsupervised log parser with llms. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2406–2429.
- [18] Yintong Huo, Yuxin Su, Cheryl Lee, and Michael R Lyu. 2023. Semparser: A semantic parser for log analytics. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 881–893.
- [19] Zhihan Jiang, Jinyang Liu, Zhuangbin Chen, Yichen Li, Junjie Huang, Yintong Huo, Pinjia He, Jiazhen Gu, and Michael R Lyu. 2024. Lilac: Log parsing using llms with adaptive parsing cache. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 137–160.
- [20] Zhihan Jiang, Jinyang Liu, Junjie Huang, Yichen Li, Yintong Huo, Jiazhen Gu, Zhuangbin Chen, Jieming Zhu, and Michael R Lyu. 2024. A large-scale evaluation for log parsing techniques: How far are we? In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 223–234.
- [21] Zhihan Jiang, Jinyang Liu, Yichen Li, Haiyu Huang, Xiao He, Tieying Zhang, Jianjun Chen, Yi Li, Rui Shi, and Michael R Lyu. 2025. LogPilot: Intent-aware and Scalable Alert Diagnosis for Large-scale Online Service Systems. *arXiv preprint arXiv:2509.25874* (2025).
- [22] Zhen Ming Jiang, Ahmed E Hassan, Parminder Flora, and Gilbert Hamann. 2008. Abstracting execution logs to execution events for enterprise applications (short paper). In *2008 The Eighth International Conference on Quality Software*. IEEE, 181–186.
- [23] Van-Hoang Le and Hongyu Zhang. 2021. Log-based anomaly detection without log parsing. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 492–504.
- [24] Van-Hoang Le and Hongyu Zhang. 2023. Log Parsing with Prompt-based Few-shot Learning. *arXiv preprint arXiv:2302.07435* (2023).
- [25] Xiaoyun Li, Hongyu Zhang, Van-Hoang Le, and Pengfei Chen. 2024. Logshrink: Effective log compression by leveraging commonality and variability of log data. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–12.
- [26] Jinyang Liu, Jieming Zhu, Shilin He, Pinjia He, Zibin Zheng, and Michael R Lyu. 2019. Logzip: Extracting hidden structures via iterative clustering for log compression. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 863–873.
- [27] Yudong Liu, Xu Zhang, Shilin He, Hongyu Zhang, Liquan Li, Yu Kang, Yong Xu, Minghua Ma, Qingwei Lin, Yingnong Dang, et al. 2022. Uniparser: A unified log parser for heterogeneous log data. In *Proceedings of the ACM Web Conference 2022 (WWW)*. 1893–1901.
- [28] Jian-Guang Lou, Qiang Fu, Shenqi Yang, Ye Xu, and Jiang Li. 2010. Mining invariants from console logs for system problem detection. In *2010 USENIX annual technical conference (USENIX ATC 10)*.
- [29] Zeyang Ma, Dong Jae Kim, and Tse-Hsun Chen. 2024. Librelog: Accurate and efficient unsupervised log parsing using open-source large language models. *arXiv preprint arXiv:2408.01585* (2024).
- [30] Adetokunbo AO Makanju, A Nur Zincir-Heywood, and Evangelos E Milios. 2009. Clustering event logs using iterative partitioning. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining (KDD)*. 1255–1264.
- [31] Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, et al. 2019. Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In *Ijcai*, Vol. 19. 4739–4745.
- [32] Suphakit Niwattanakul, Jatsada Singthongchai, Ekkachai Naenudorn, and Supachanun Wanapu. 2013. Using of Jaccard coefficient for keywords similarity. In *Proceedings of the international multicongress of engineers and computer scientists*, Vol. 1. 380–384.
- [33] Paolo Notaro, Sorous Haeri, Jorge Cardoso, and Michael Gerndt. 2023. LogRule: Efficient Structured Log Mining for Root Cause Analysis. *IEEE Transactions on Network and Service Management* (2023).
- [34] David Ohana. 2020. A blog about Drain usage in IBM Cloud. <https://developer.ibm.com/blogs/how-mining-log-templates-can-help-ai-ops-in-cloud-scale-data-centers/>. Online; Accessed: 2026-03-16.
- [35] OpenAI. [n. d.]. GPT-5.2. <https://platform.openai.com/docs/models/gpt-5.2>. Online; Accessed: 2026-03-16.
- [36] Daan Schipper, Mauricio Aniche, and Arie van Deursen. 2019. Tracing back log data to its log statement: from research to practice. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 545–549.
- [37] Issam Sedki, Abdelwahab Hamou-Lhadj, Othmane Ait-Mohamed, and Mohammed A Shehab. 2022. An Effective Approach for Parsing Large Log Files. In *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 1–12.
- [38] Shiwen Shan, Yintong Huo, Yuxin Su, Yichen Li, Dan Li, and Zibin Zheng. 2024. Face it yourselves: An llm-based two-stage strategy to localize configuration errors via logs. In *Proceedings of the 33rd ACM SIGSOFT international symposium on software testing and analysis*. 13–25.
- [39] Keiichi Shima. 2016. Length matters: Clustering system log messages using length of words. *arXiv preprint arXiv:1611.03213* (2016).
- [40] Donghwan Shin, Zanis Ali Khan, Domenico Bianculli, and Lionel Briand. 2021. A theoretical framework for understanding the relationship between log parsing and anomaly detection. In *International Conference on Runtime Verification*. Springer, 277–287.
- [41] Risto Vaarandi. 2003. A data clustering algorithm for mining patterns from event logs. In *Proceedings of the 3rd IEEE Workshop on IP Operations & Management (IPOM)*(IEEE Cat. No. 03EX764). Ieee, 119–126.
- [42] Risto Vaarandi and Mauno Pihelgas. 2015. Logcluster-a data clustering and pattern mining algorithm for event logs. In *2015 11th international conference on network and service management (CNSM)*. IEEE, 1–7.

- [43] Lingzhi Wang, Nengwen Zhao, Junjie Chen, Pinnong Li, Wenchi Zhang, and Kaixin Sui. 2020. Root-cause metric location for microservice systems via log anomaly detection. In *2020 IEEE international conference on web services (ICWS)*. IEEE, 142–150.
- [44] Teng Wang, Xiaodong Liu, Shanshan Li, Xiangke Liao, Wang Li, and Qing Liao. 2018. MisconfDoctor: diagnosing misconfiguration via log-based configuration testing. In *2018 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 1–12.
- [45] Thorsten Wittkopp, Philipp Wiesner, and Odej Kao. 2024. Logrca: Log-based root cause analysis for distributed services. In *European Conference on Parallel Processing*. Springer, 362–376.
- [46] Pei Xiao, Tong Jia, Chiming Duan, Minghua He, Weijie Hong, Xixuan Yang, Yihan Wu, Ying Li, and Gang Huang. 2025. Clslog: Collaborating large and small models for log-based anomaly detection. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 686–690.
- [47] Yi Xiao, Van-Hoang Le, and Hongyu Zhang. 2024. free: Towards more practical log parsing with large language models. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 153–165.
- [48] Weiyuan Xu, Juntao Luo, Tao Huang, Kaixin Sui, Jie Geng, Qijun Ma, Isami Akasaka, Xiaoxue Shi, Jing Tang, and Peng Cai. 2025. LogSage: An LLM-based framework for CI/CD failure detection and remediation with industrial validation. *arXiv preprint arXiv:2506.03691* (2025).
- [49] Siyu Yu, Pinjia He, Ningjiang Chen, and Yifan Wu. 2023. Brain: Log parsing with bidirectional parallel tree. *IEEE Transactions on Services Computing* 16, 5 (2023), 3224–3237.
- [50] Siyu Yu, Yifan Wu, Ying Li, and Pinjia He. 2024. Unlocking the power of numbers: Log compression via numeric token parsing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 919–930.
- [51] Ding Yuan, Haoxue Mai, Weiwei Xiong, Lin Tan, Yuanyuan Zhou, and Shankar Pasupathy. 2010. Sherlog: error diagnosis by connecting clues from run-time logs. In *Proceedings of the fifteenth International Conference on Architectural support for programming languages and operating systems*. 143–154.
- [52] Chenbo Zhang, Wenying Xu, Jinbu Liu, Lu Zhang, Guiyang Liu, Jihong Guan, Qi Zhou, and Shuigeng Zhou. 2025. SemanticLog: Towards Effective and Efficient Large-Scale Semantic Log Parsing. *IEEE Transactions on Software Engineering* (2025).
- [53] Xu Zhang, Yong Xu, Qingwei Lin, Bo Qiao, Hongyu Zhang, Yingnong Dang, Chunyu Xie, Xinsheng Yang, Qian Cheng, Ze Li, et al. 2019. Robust log-based anomaly detection on unstable log data. In *Proceedings of the 2019 27th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*. 807–817.
- [54] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Dewei Liu, Qilin Xiang, and Chuan He. 2019. Latent error prediction and fault localization for microservice applications by learning from system trace logs. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE)*. 683–694.
- [55] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R Lyu. 2023. Loghub: A large collection of system log datasets for ai-driven log analytics. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 355–366.
- [56] Jieming Zhu, Shilin He, Jinyang Liu, Pinjia He, Qi Xie, Zibin Zheng, and Michael R Lyu. 2019. Tools and benchmarks for automated log parsing. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 121–130.